

CSE 579 Homework 1

Hisham Bhatti

1 Code Overview

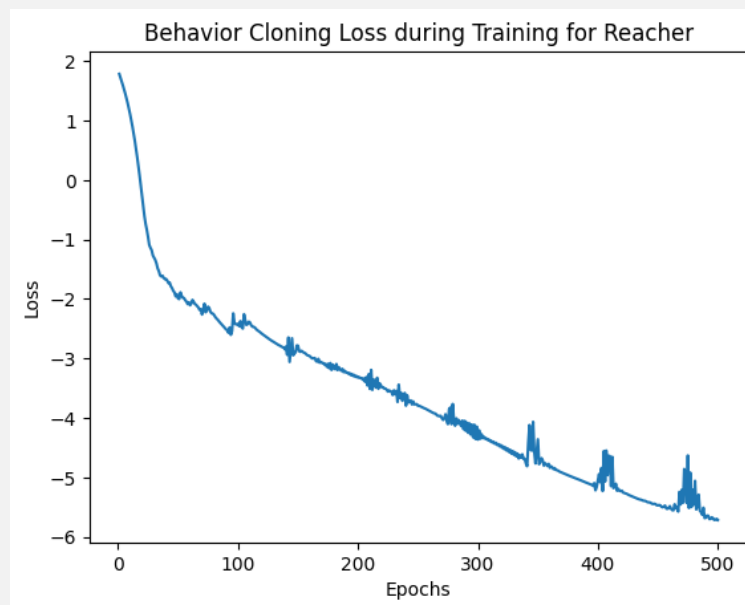
2 Behavior Cloning

1. Fill in the blanks in the code marked with `TODO` in the `simulate_policy_bs` function in `bc.py`

See Gradescope submission for code.

2. Plot the loss during the training with default hyper-parameters. Report the success rate and average reward using the `evaluate` function that is provided to you.

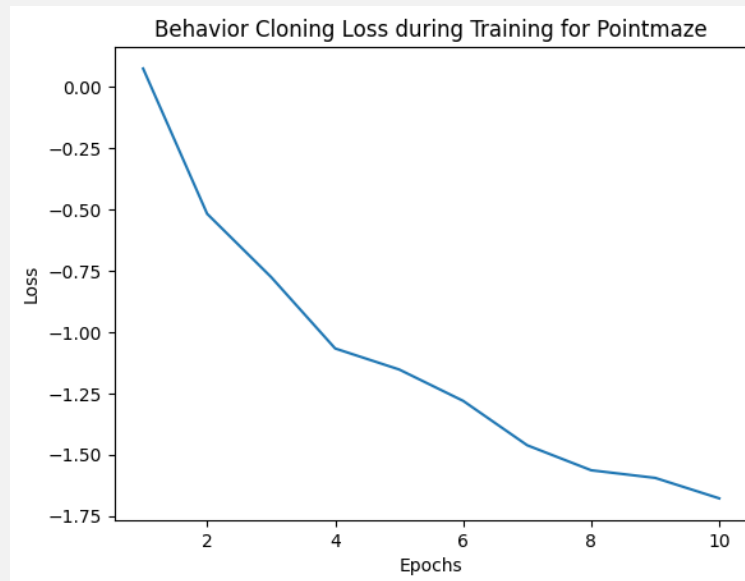
Environment: Reacher



Success rate: 0.31

Average reward (success only): -12.62037562981233

Average reward (all): -13.600105860683119

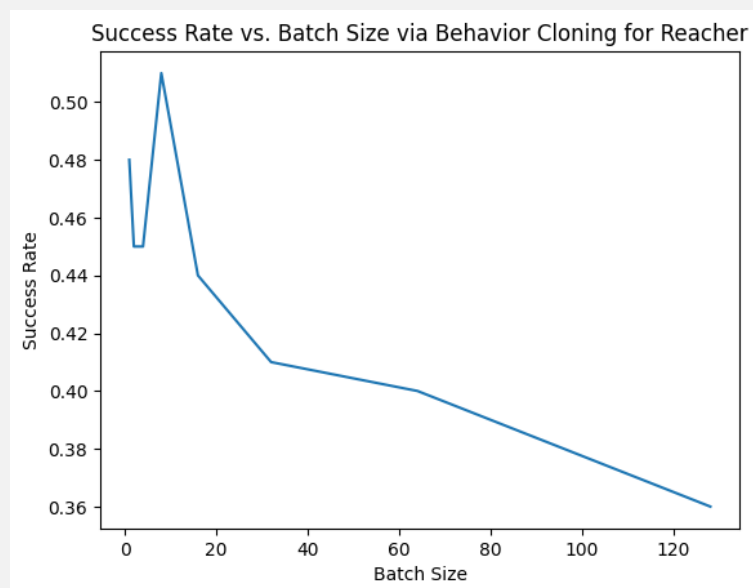
Environment: PointMaze

Success rate: 0.17

Average reward (success only): 1.0

Average reward (all): 0.17

3. Experiment with one set of hyperparameters that affects the performance of the behavioral cloning agent, such as the amount of training steps, the amount of expert data provided, or something that you come up with yourself. For one of the tasks used in the previous question, show a graph of how the BC agent's performance varies with the value of this hyperparameter. In the caption for the graph, state the hyperparameter and a brief rationale for why you chose it.

Hyperparameter: Batch Size

I chose to adjust *batch size* and measure success rate given our discussion in class of the importance of a large batch size to speed up training. Larger batch sizes dramatically increase vectorization and speed up training, but can often converge to a sharp minima that might not be global. In this experiment, we can clearly see a decrease in accuracy as batches become larger, signaling that the training converged to worse values.

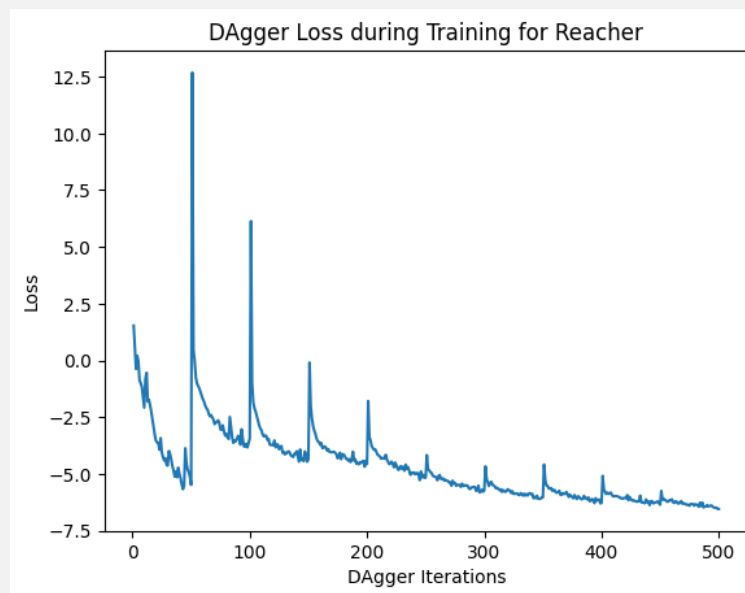
3 DAgger

1. Fill in the blanks in the code marked with `TODO` in the `simulate_policy_dagger` function in `dagger.py`.

See Gradescope submission for code.

2. Plot the loss during training with default hyper-parameters. Report the success rate and average reward using the `evaluate` function that is provided to you.

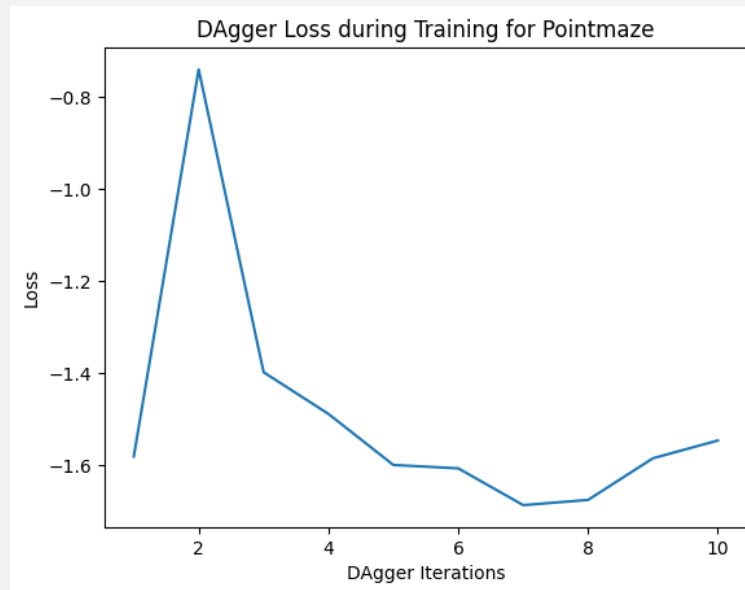
Environment: Reacher



Success rate: 1.0

Average reward (success only): -4.355398707310565

Average reward (all): -4.355398707310565

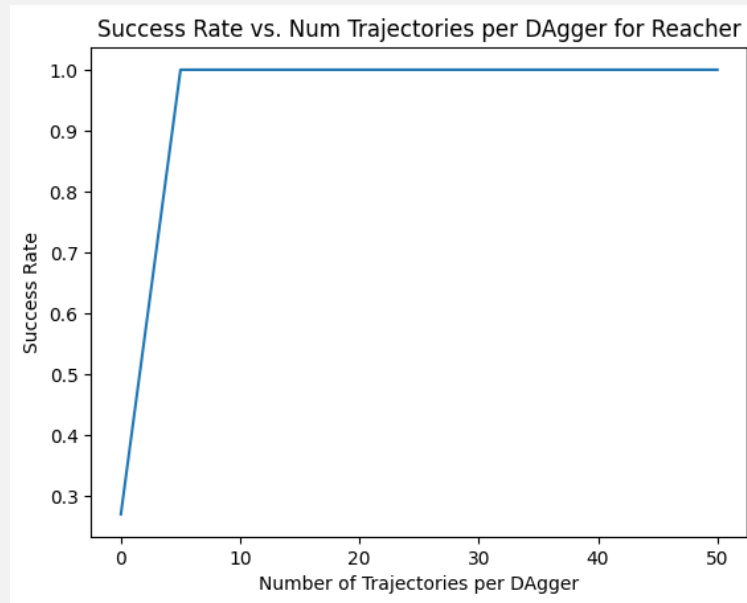
Environment: PointMaze**Success rate:** 0.73**Average reward (success only):** 1.0**Average reward (all):** 0.73

3. Compare the success rate and reward of the DAgger policy with the behavior cloning policy and explain why DAgger performs better.

For both tasks, the DAgger policy had a much larger success rate and average reward (both success only and all) compared to standard behavior cloning. The DAgger policy performs better since it dynamically corrects errors and prevents drift. In standard behavior cloning, the trained policy is susceptible to compounding errors as small deviations in expected behavior can amplify over time. By providing expert labelling online, DAgger aims to correct for these errors at inference time, leading to robust, stable behavior.

4. Experiment with one set of hyperparameters that affects the performance of the agent, such as the amount of training steps, the amount of expert data provided, or something that you come up with yourself. For one of the tasks used in the previous question, show a graph of how the agent's performance varies with the value of this hyperparameter. In the caption for the graph, state the hyperparameter and a brief rationale for why you chose it.

Hyperparameters: Number of Trajectories per DAgger



I chose to adjust the *number of trajectories* to be relabeled per DAgger iteration since I wanted to understand how many is sufficient for high performance in this dataset. While DAgger leads to high performance, it often is expensive in terms of both time, compute, and human effort. From the graph, we can see that only 5 trajectories relabeled per DAgger is enough for high performance; any more does not provide much benefit.

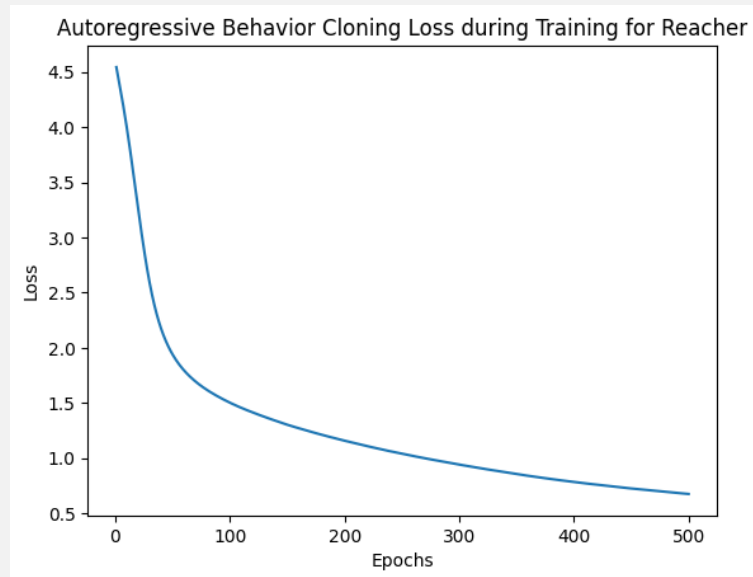
4 Autoregressive Model

1. Fill in the blanks in the code marked with TODO in the PolicyAutoRegressiveModel class in `utils.py`.

See Gradescope submission for code.

2. Plot the loss during training using behavior cloning and DAgger, with default hyperparameters for the pointmaze environment. Report the success rate and average reward using the evaluate function that is provided to you.

Environment: Reacher

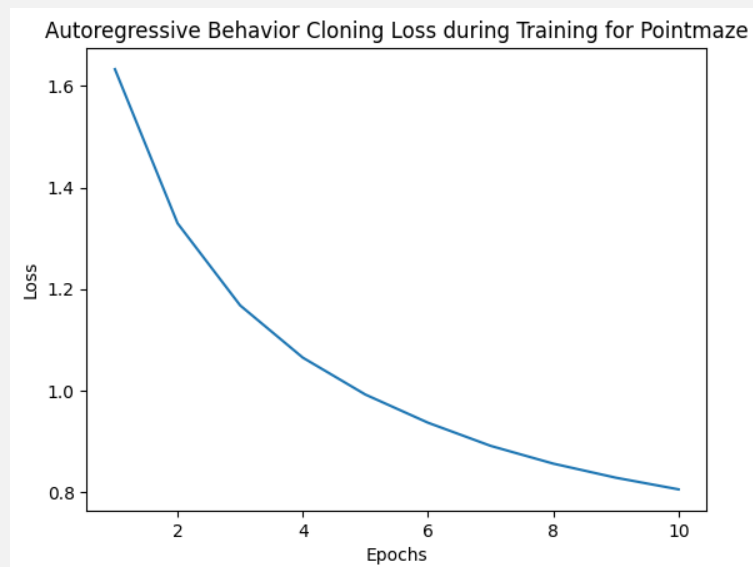


Success rate: 0.16

Average reward (success only): -8.599067272045591

Average reward (all): -10.915172321963519

Environment: PointMaze



Success rate: 0.82

Average reward (success only): 1.0

Average reward (all): 0.82

3. Compare the success rate and reward of your autoregressive policies with training with Gaussian policies, and explain your observation.

For the Reacher environment, the autoregressive policy has a lower success rate and average reward than the Gaussian policies (behavior cloning and DAgger). However, for the Pointmaze environment, the success rate and average reward of the autoregressive policy is much higher.

I think the reason for this difference in performance comes down to the complexity of each task, and how well they discretize. The PointMaze environment is a more complex task than the Reacher, requiring long-horizon navigation. As such, a policy like autoregressive might perform better as it trains networks from a history of actions, rather than the Gaussian policy which outputs single-step actions. Furthermore, the Reacher task involves a mechanical arm, and feels more continuous than the PointMaze task, which can be thought of as moving a ball 4 directions. An autoregressive policy is inherently discrete, and therefore might more easily adapt to the PointMaze environment.

5 Diffusion Policies [Extra Credit]